COMPLETE DEVELOPER LEARNING ROADMAP

11 Unique Projects — From Zero to Full-Stack Developer

A Comprehensive Guide for Beginners | ~18 Months to Complete All Projects

---

How to Use This Guide

This roadmap is structured in two parts: (1) a Master Learning Timetable that builds your foundational skills, and (2) a deep-dive guide for each of the 11 projects — including every language, framework, algorithm, and tool you'll need, plus curated free and paid learning resources. Follow the phases in order. Do not skip foundations — every advanced project here depends on them.

The 11 Projects at a Glance

#	Project	Category	Est. Duration	Difficulty
1	Basic Search Engine	Search & NLP	3–4 weeks	⭑⭑
2	AI-Powered Search Engine	AI / RAG	4–6 weeks	⭑⭑⭑⭑
3	Food Delivery App with Reels	Full-Stack / Mobile	4–5 weeks	⭑⭑⭑
4	Facial Recognition Photo Organizer	Computer Vision / ML	4–5 weeks	⭑⭑⭑⭑
5	Anti-Pollution Route Planner	Maps / APIs / Data	3–4 weeks	⭑⭑⭑
6	Managed Service Infra Orchestration Platform	DevOps / Cloud	5–6 weeks	⭑⭑⭑⭑⭑
7	Scribble Clone (Multiplayer Drawing)	Realtime / WebSockets	3–4 weeks	⭑⭑⭑
8	Banana Trading Company (Game)	Game Design / Full-Stack	4–5 weeks	⭑⭑⭑⭑
9	LLM Minecraft (Strategy Game)	AI / LLM Integration	4–5 weeks	⭑⭑⭑⭑
10	AI Workflow Automation Tool	LLM Pipelines / SaaS	5–6 weeks	⭑⭑⭑⭑⭑
11	PR Review Tool (GitHub App)	DevTools / AI / GitHub	4–5 weeks	⭑⭑⭑⭑

PART 1: MASTER LEARNING TIMETABLE

~18 Months | Phases 1–6 → Build All 11 Projects

The timetable below is divided into 6 phases. Phases 1–3 are pure learning (foundations). Phases 4–6 involve building the projects while learning project-specific tools. Each phase assumes 2–3 hours of study per day, 5 days/week.

Phase / Week	Topics & Skills	Projects / Milestones	Resources
Phase 1 Weeks 1–6 FOUNDATIONS	• Python (syntax, OOP, modules, venv, pip) • JavaScript & TypeScript (ES6+, async/await, types) • HTML & CSS (semantic HTML, Flexbox, Grid) • Git & GitHub (branches, PRs, commits) • Command Line / Bash scripting • Data Structures & Algorithms basics • SQL basics (SELECT, JOIN, indexes)	✓ Build a CLI to-do list in Python ✓ Build a static portfolio site ✓ Push all projects to GitHub ✓ Write 5 SQL queries on a sample DB	CS50P (Harvard, free) The Odin Project freeCodeCamp.org SQLZoo.net Pro Git Book (free)
Phase 2 Weeks 7–12 BACKEND & WEB	• Node.js + Express.js (REST APIs, middleware) • Python FastAPI / Flask (REST APIs) • PostgreSQL + Prisma ORM • MongoDB + Mongoose • Authentication: JWT, OAuth2, bcrypt • Docker basics (Dockerfile, Compose) • HTTP, REST, WebSockets fundamentals • Redis (caching, pub/sub)	✓ Build a full CRUD REST API with auth ✓ Containerize the API with Docker ✓ Connect PostgreSQL + Redis ✓ WebSocket chat demo	Node.js official docs FastAPI official docs Docker getting started Prisma docs Redis University (free)
Phase 3 Weeks 13–18 FRONTEND & CLOUD	• React.js (hooks, context, routing) • Next.js (SSR, SSG, API routes) • Tailwind CSS • React Query / SWR for data fetching • AWS / GCP basics (S3, EC2, Lambda) • nginx basics (reverse proxy, static files) • CI/CD basics (GitHub Actions) • Kubernetes intro (pods, deployments)	✓ Build a full-stack Next.js app with DB ✓ Deploy to AWS/Vercel ✓ Set up a CI/CD pipeline ✓ Run a Kubernetes pod locally	React official docs (react.dev) Next.js official docs AWS Free Tier tutorials Kubernetes.io interactive tutorial Tailwind CSS docs
Phase 4 Weeks 19–28 PROJECTS 1–5 (Search, Delivery, CV)	• NLP: TF-IDF, BM25, tokenization (NLTK, spaCy) • Vector embeddings (OpenAI / sentence-transformers)	✓ Project 1: Basic Search Engine ✓ Project 2: AI-Powered Search Engine	LangChain docs OpenAI API docs OpenCV Python tutorials DeepFace GitHub

	• RAG architecture (LangChain, LlamaIndex) • LLM APIs (OpenAI, Anthropic, Gemini) • Computer Vision (OpenCV, face_recognition, DeepFace) • Real-time video processing • Maps APIs (Google Maps, OpenRouteService) • Air quality APIs (OpenAQ, WAQI)	✓ Project 3: Food Delivery App with Reels ✓ Project 4: Facial Recognition Photo Organizer ✓ Project 5: Anti-Pollution Route Planner	Google Maps Platform docs
Phase 5 Weeks 29–36 PROJECTS 6–9 (Infra, Games, AI)	• Advanced Docker & Kubernetes (operators, CRDs) • Infrastructure as Code (Terraform basics) • WebSockets in depth (Socket.io) • Canvas API / Phaser.js (game rendering) • Game state management, real-time sync • LLM multi-agent orchestration • Dynamic pipeline design for LLMs • GitHub OAuth, GitHub Apps, GitHub API	✓ Project 6: Managed Service Infra Orchestration ✓ Project 7: Scribble Clone ✓ Project 8: Banana Trading Company ✓ Project 9: LLM Minecraft	Terraform getting started Socket.io docs Phaser.js docs LangGraph docs GitHub Apps docs
Phase 6 Weeks 37–44 PROJECTS 10–11 (SaaS & DevTools)	• LLM pipeline orchestration (Dify, LangFlow, custom) • Background job queues (BullMQ, Celery) • Webhook systems & event-driven architecture • Multi-LLM routing and fallback logic • Code analysis: AST parsing, static analysis • PR diff parsing (GitHub API: /pulls/:id/files) • Modular plugin architecture • Deployment: Railway, Render, AWS Lambda	✓ Project 10: AI Workflow Automation Tool ✓ Project 11: PR Review Tool ✓ Deploy all 11 projects publicly ✓ Write READMEs for all projects ✓ Build portfolio site showcasing all work	BullMQ docs Dify.ai docs GitHub REST API docs AWS Lambda getting started Render.com docs

Total Time Estimate

Study Hours/Day	2–3 hours (beginner pace), 4–5 hours (accelerated)
Days per Week	5 days (weekdays); weekends optional for projects
Phases 1–3 (Foundations)	~18 weeks (~4.5 months)
Phases 4–6 (Projects)	~26 weeks (~6.5 months)
Total Estimate	~44 weeks (11 months accelerated) to 18 months at beginner pace
Absolute Minimum	If you already know Python/JS basics, start at Phase 2 (~12 months total)

PART 2: PROJECT-BY-PROJECT GUIDE

Full tech stack, algorithms, and resources for each project

Project 1: Basic Search Engine

A web crawler + indexer + search engine with autocomplete and analytics

Tech Stack & Tools

Language	Python 3.10+
Web Crawling	Scrapy or BeautifulSoup4 + requests
Text Processing	NLTK, spaCy (tokenization, stop word removal, stemming)
Search Algorithms	TF-IDF (sklearn), BM25 (rank_bm25 library)
Vectorization	scikit-learn TfidfVectorizer, scipy sparse matrices
Database	PostgreSQL (indexed documents), Elasticsearch (optional)
Frontend	React.js + Tailwind CSS
Backend API	FastAPI (Python) or Express.js (Node.js)
Autocomplete	Prefix tree (Trie) data structure, Redis sorted sets
Caching	Redis (cache popular queries)
Pagination	Cursor-based or offset pagination
Analytics	Logging queries to PostgreSQL, simple dashboard

Key Algorithms & Concepts

- TF-IDF (Term Frequency–Inverse Document Frequency): Scores word importance in docs
- BM25 (Okapi BM25): Probabilistic ranking function — superior to plain TF-IDF
- Trie (Prefix Tree): O(L) autocomplete lookups where L = query length
- Inverted Index: Maps each term → list of document IDs containing that term
- Cosine Similarity: Measures angle between document and query vectors
- Stop Word Removal & Stemming: Porter Stemmer / Lemmatization via NLTK
- PageRank (optional): Graph algorithm to rank crawled pages by link authority

Build Steps

1. Set up Python project: virtualenv, requirements.txt, folder structure
2. Build the crawler: Scrapy spider targeting a seed URL, respect robots.txt
3. Process crawled HTML: Strip tags, tokenize, remove stop words, stem
4. Build inverted index: Python dict or store in PostgreSQL with GIN indexes

5. Implement TF-IDF scoring with sklearn, then add BM25 for comparison
6. Build Trie for autocomplete — store in Redis sorted sets for persistence
7. Create FastAPI REST endpoint: GET /search?q=query returns ranked docs
8. Build React frontend: search bar with autocomplete dropdown
9. Add Redis caching layer for top-1000 queries
10. Add pagination, analytics logging, and a simple dashboard page

Learning Resources

TF-IDF Explained	https://www.tfidf.com — visual explanation
BM25 Library	https://pypi.org/project/rank-bm25
Scrapy Tutorial	https://docs.scrapy.org/en/latest/intro/tutorial.html
NLTK Book (free)	https://www.nltk.org/book
FastAPI Docs	https://fastapi.tiangolo.com
Elasticsearch Guide	https://www.elastic.co/guide/en/elasticsearch/reference/current/getting-started.html
Redis Data Structures	https://redis.io/docs/data-types
CS50 DSA (free)	https://cs50.harvard.edu/x

Project 2: AI-Powered Search Engine (RAG System)

Semantic search with vector embeddings, LLM-powered answers, and recursive querying

Tech Stack & Tools

Language	Python 3.10+
Vector Embeddings	OpenAI text-embedding-3-small, or sentence-transformers (free)
Vector DB	Pinecone (cloud) or pgvector (PostgreSQL extension) or Chroma (local)
LLM for Answers	OpenAI GPT-4o, Anthropic Claude, or Google Gemini API
RAG Framework	LangChain or LlamaIndex
Reranking	Cohere Rerank API or cross-encoder from sentence-transformers
Backend	FastAPI + Python
Frontend	Next.js 14 + Tailwind CSS + shadcn/ui
Real-time Streaming	Server-Sent Events (SSE) or WebSockets for LLM streaming
Database	PostgreSQL for metadata, Pinecone/Chroma for vectors
Recursive Querying	LangChain's ReAct agent or custom query decomposition chain

Key Algorithms & Concepts

- Retrieval-Augmented Generation (RAG): Retrieve relevant docs, inject into LLM prompt
- Semantic Search: Embed queries + docs into high-dimensional vector space, compare with cosine similarity
- Approximate Nearest Neighbor (ANN): HNSW algorithm (used by Pinecone/Chroma) for fast vector search
- Query Decomposition: Break complex queries into sub-questions (LangChain multi-query retriever)
- Contextual Compression: Re-rank and compress retrieved chunks before sending to LLM
- Recursive Retrieval: Answer sub-questions independently, then synthesize a final answer
- Chunking Strategies: Fixed-size, sentence-based, or semantic chunking of documents
- Hypothetical Document Embeddings (HyDE): Embed a hypothetical answer to improve retrieval

Build Steps

11. Build the document ingestion pipeline: crawl → chunk → embed → store in vector DB
12. Implement semantic search: embed user query, find top-k nearest chunks via ANN search
13. Add a reranking step: cross-encoder scores retrieved chunks for relevance
14. Build LLM answer generation: stuff retrieved chunks into a prompt, call GPT/Claude
15. Add streaming: stream LLM tokens to frontend via SSE for real-time feel
16. Implement recursive querying: decompose queries → answer each sub-part → synthesize

17. Add source citation: return which documents were used in each answer
18. Build the frontend: Next.js app with streaming text, source cards, follow-up suggestions
19. Add rate limiting, error handling, and fallback to keyword search when needed

Learning Resources

LangChain RAG Tutorial	https://python.langchain.com/docs/tutorials/rag
LlamaIndex Docs	https://docs.llamaindex.ai
Pinecone Quickstart	https://docs.pinecone.io/guides/getting-started/quickstart
pgvector GitHub	https://github.com/pgvector/pgvector
sentence-transformers	https://www.sbert.net
OpenAI Embeddings Guide	https://platform.openai.com/docs/guides/embeddings
RAG Survey Paper (arxiv)	https://arxiv.org/abs/2312.10997
DeepLearning.AI RAG Course (free)	https://learn.deeplearning.ai/courses/building-and-evaluating-advanced-rag

Project 3: Food Delivery App with Reels

Instagram + Zomato hybrid: short food discovery videos + delivery platform

Tech Stack & Tools

Language	TypeScript (full-stack)
Frontend	Next.js 14 (App Router), Tailwind CSS, shadcn/ui
Mobile (optional)	React Native + Expo for iOS/Android app
Backend	Node.js + Express.js or Next.js API Routes
Database	PostgreSQL + Prisma ORM
File/Video Storage	AWS S3 (video uploads), CloudFront CDN
Video Processing	FFmpeg (thumbnail generation, compression)
Auth	NextAuth.js (OAuth with Google/Facebook) or Clerk
Real-time	Socket.io for live order tracking
Maps	Google Maps JavaScript API (delivery tracking)
Payments	Stripe API (card payments)
Push Notifications	Firebase Cloud Messaging (FCM) or OneSignal
Search	PostgreSQL full-text search or Algolia

Key Algorithms & Concepts

- Feed Algorithm: Collaborative filtering or simple recency + engagement scoring for Reels
- Video Streaming: Chunked HTTP streaming or HLS (HTTP Live Streaming) for short videos
- Geospatial Queries: PostGIS extension for PostgreSQL — find nearby restaurants by radius
- Order State Machine: Pending → Confirmed → Preparing → Out for Delivery → Delivered
- Rate Limiting: Token bucket algorithm to prevent API abuse
- Infinite Scroll: Cursor-based pagination for the Reels feed
- Optimistic UI Updates: Update UI before server confirmation for snappy feel

Build Steps

20. Design database schema: Users, Restaurants, Menu Items, Orders, Reels, Reviews
21. Build restaurant CRUD API with location data (PostGIS latitude/longitude)
22. Implement video upload: accept file → upload to S3 → process with FFmpeg → serve via CloudFront
23. Build the Reels feed: vertical scroll with autoplay, like, save, redirect-to-restaurant
24. Implement the cart and checkout flow with Stripe payment integration
25. Add real-time order tracking with Socket.io: driver location updates every 10 seconds

26. Build restaurant owner dashboard: menu management, order management, analytics
27. Add search with filtering by cuisine, price range, rating, and delivery time
28. Implement push notifications for order status changes
29. Deploy: Vercel (frontend), Railway/Render (backend), AWS S3 (media)

Learning Resources

Next.js 14 Course	https://nextjs.org/learn (official, free)
Prisma with PostgreSQL	https://www.prisma.io/docs/getting-started
Stripe Payment Integration	https://stripe.com/docs/payments/quickstart
AWS S3 with Node.js	https://docs.aws.amazon.com/sdk-for-javascript/v3/developer-guide/s3-example-creating-buckets.html
FFmpeg Tutorial	https://ffmpeg.org/documentation.html
Socket.io with React	https://socket.io/how-to/use-with-react
PostGIS Intro	https://postgis.net/documentation/getting_started
React Native + Expo	https://docs.expo.dev/tutorial/introduction

Project 4: Facial Recognition Photo Organizer

Auto-group photos by person using ML face recognition with front camera support

Tech Stack & Tools

Language	Python (backend ML), TypeScript (frontend)
Face Detection	OpenCV (Haar cascades or DNN-based detector)
Face Recognition	face_recognition library (dlib) or DeepFace
Deep Learning Models	ArcFace, FaceNet, or VGGFace2 (via DeepFace)
Backend API	FastAPI (Python) — handles ML inference
Frontend	Next.js + React Webcam (front camera capture)
Database	PostgreSQL (metadata) + vector store for face embeddings
File Storage	AWS S3 or local filesystem (development)
Pipeline Infra	Celery + Redis (async photo processing queue)
Similarity Search	FAISS (Facebook AI Similarity Search) for fast face matching
Privacy	All processing server-side; no raw face images sent to third-party APIs

Key Algorithms & Concepts

- Face Detection: Viola-Jones algorithm (Haar cascades) or MTCNN (deep learning)
- Face Embedding: Deep neural network maps face → 128-512 dimensional vector (FaceNet: 128D)
- Face Verification: Euclidean distance or cosine similarity between embeddings (threshold ~0.6)
- Clustering: DBSCAN algorithm to group faces without knowing number of people in advance
- Transfer Learning: Pre-trained models (ArcFace/FaceNet) fine-tuned on your use case
- FAISS Index: Inverted file index (IVF) for sub-linear time nearest-neighbor search
- Data Augmentation: Flip, rotate, brightness changes to improve model robustness
- Anti-Spoofing (optional): Depth map or texture analysis to prevent photo attacks

Build Steps

30. Set up Python backend: FastAPI, Celery, Redis, FAISS, and face_recognition/DeepFace
31. Build face detection endpoint: upload photo → detect faces → return bounding boxes
32. Build face embedding endpoint: detected face crop → 128D vector
33. Store embeddings in FAISS index; also store in PostgreSQL for persistence
34. Implement clustering: DBSCAN on stored embeddings to auto-group photos by person
35. Build front camera capture: React Webcam component → send frame to API for live detection
36. Build the photo organizer UI: grid view → click person → see all photos of that person

37. Add manual tagging: user can name identified clusters (Person 1 → 'Mom')
38. Implement async pipeline: new photo upload → Celery task → embed → cluster update
39. Add privacy mode: process locally in browser with face-api.js (TensorFlow.js)

Learning Resources

face_recognition library	https://github.com/ageitgey/face_recognition
DeepFace GitHub	https://github.com/serengil/deepface
OpenCV Python Tutorials	https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
FAISS Getting Started	https://faiss.ai/index.html
DBSCAN Explained (scikit-learn)	https://scikit-learn.org/stable/modules/clustering.html#dbscan
face-api.js (browser)	https://github.com/justadudewhohacks/face-api.js
FastAPI Background Tasks	https://fastapi.tiangolo.com/tutorial/background-tasks
Celery + Redis Tutorial	https://docs.celeryq.dev/en/stable/getting-started/first-steps-with-celery.html

Project 5: Anti-Pollution Route Planner

Real-time AQI + weather + traffic data to suggest the least polluted routes

Tech Stack & Tools

Language	Python (data aggregation), TypeScript (frontend)
Mapping	Google Maps JavaScript API or Mapbox GL JS
Routing API	Google Directions API, OpenRouteService, or Mapbox Directions
AQI Data	OpenAQ API, WAQI (World Air Quality Index) API, IQAir API
Weather Data	OpenWeatherMap API or Open-Meteo (free, no key needed)
Traffic Data	Google Maps Traffic Layer or TomTom Traffic API
Backend	FastAPI or Flask (Python)
Frontend	Next.js + React + Mapbox GL JS or Google Maps React
Database	PostgreSQL + PostGIS (geospatial data)
Notifications	Web Push API or email via SendGrid for pollution alerts
ML (optional)	Linear regression / XGBoost to predict AQI from historical data

Key Algorithms & Concepts

- Dijkstra's Algorithm: Classic shortest path; modified here to minimize pollution score (not distance)
- A* Algorithm: Heuristic-based shortest path — faster than Dijkstra for maps
- Graph Representation: Road network as weighted directed graph (nodes=intersections, edges=roads)
- AQI Calculation: US EPA formula: AQI = (Ihigh Ilow)/(Chigh-Clow) × (C-Clow) + Ilow
- Weighted Multi-Objective Routing: Combine AQI + distance + time with user-adjustable weights
- Geospatial Interpolation: IDW (Inverse Distance Weighting) to estimate AQI at road segments
- Time-Series Prediction: SARIMA or Prophet model for hourly AQI forecasting
- Heatmap Rendering: Deck.gl or Google Maps HeatmapLayer to visualize pollution hotspots

Build Steps

40. Register for AQI API keys: WAQI, OpenAQ, and OpenWeatherMap
41. Build a data aggregation service: fetch AQI stations near a city, store in PostgreSQL with PostGIS
42. Implement AQI interpolation: assign pollution score to each road segment via IDW
43. Build the routing engine: query Directions API for multiple route options, score each
44. Create ranking algorithm: weighted score = α×distance + β×AQI + γ×travelTime
45. Build the map UI: Mapbox GL JS with pollution heatmap overlay and route comparison

46. Add route comparison panel: show 3 routes with distance, time, and AQI score
47. Add notifications: alert users when AQI on their commute exceeds threshold
48. Add jogger/cyclist mode: different default weights (care more about AQI than time)
49. Build export: share clean route as link or GPX file for GPS devices

Learning Resources

WAQI API Docs	https://aqicn.org/api
OpenAQ Docs	https://docs.openaq.org
OpenRouteService Docs	https://openrouteservice.org/dev/#/api-docs
Mapbox GL JS Docs	https://docs.mapbox.com/mapbox-gl-js/guides
Google Maps Routes API	https://developers.google.com/maps/documentation/routes
PostGIS Workshop (free)	https://postgis.net/workshops/postgis-intro
Deck.gl Heatmap Layer	https://deck.gl/docs/api-reference/aggregation-layers/heatmap-layer
Prophet Forecasting (free)	https://facebook.github.io/prophet/docs/quick_start.html

Project 6: Managed Service Infra Orchestration Platform

A Vercel-like platform: deploy static sites from GitHub repos or ZIP files to S3 via containers

Tech Stack & Tools

Language	Node.js (TypeScript) backend, React frontend
Core Infrastructure	Docker (build & package sites), nginx (reverse proxy/routing)
Cloud Platform	AWS S3 (static hosting), AWS EC2 or ECS (container execution)
Orchestration	AWS ECS + Fargate, or self-hosted Kubernetes (k3s)
Job Queue	BullMQ + Redis (deployment job queue with retry logic)
Auth	GitHub OAuth App (login with GitHub)
Database	PostgreSQL (users, projects, deployment records)
DNS/Routing	nginx + wildcard subdomains for {project}.yourdomain.com
CI Trigger	GitHub Webhooks (auto-deploy on push)
Frontend	Next.js + Tailwind CSS + shadcn/ui
Logging	Winston + CloudWatch Logs or self-hosted Loki + Grafana
IaC (optional)	Terraform for provisioning AWS resources

Key Algorithms & Concepts

- Container Build Pipeline: GitHub repo → clone → Docker build → extract /dist or /build → upload to S3
- Event-Driven Queue: BullMQ FIFO queue processes deployments; failed jobs retry with exponential backoff
- Reverse Proxy Routing: nginx maps subdomain → S3 static site or running container port
- Webhook Processing: Validate GitHub HMAC-SHA256 signature → parse push event → enqueue deployment
- Blue-Green Deployment: Deploy new version alongside old, swap traffic atomically (zero downtime)
- Rolling Updates: Replace container instances one by one to avoid full downtime
- Idempotent Operations: Each deployment has a unique ID; duplicate webhooks won't double-deploy
- Pre-signed S3 URLs: Securely upload build artifacts without exposing AWS credentials

Build Steps

50. Set up AWS account: create IAM user, S3 bucket with static hosting, CloudFront distribution
51. Build GitHub OAuth login: register GitHub OAuth App, implement callback, store tokens
52. Build the deployment API: POST /deploy accepts GitHub repo URL or ZIP, enqueues job

53. Write the Docker build runner: clone repo → npm install + npm build → copy output → upload to S3
54. Set up BullMQ: Redis-backed queue, worker process picks up jobs, updates DB with status
55. Implement nginx routing: wildcard subdomain config → proxy to S3 or running container
56. Add GitHub Webhook receiver: verify signature, parse push event, auto-trigger deploy
57. Build the dashboard UI: project list, deployment history, logs viewer, one-click deploy
58. Implement retry logic and failure notifications (email/Slack webhook)
59. Add Terraform scripts to provision all AWS resources reproducibly

Learning Resources

BullMQ Docs	https://docs.bullmq.io
AWS S3 Static Hosting	https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html
GitHub Webhooks Guide	https://docs.github.com/en/webhooks/about-webhooks
Docker Multi-stage Builds	https://docs.docker.com/build/building/multi-stage
nginx Reverse Proxy Guide	https://nginx.org/en/docs/beginners_guide.html
Terraform AWS Tutorial	https://developer.hashicorp.com/terraform/tutorials/aws-get-started
Kubernetes.io Tutorial	https://kubernetes.io/docs/tutorials/kubernetes-basics
Loki + Grafana Setup	https://grafana.com/docs/loki/latest/get-started/quick-start

Project 7: Scribble Clone — Real-Time Multiplayer Drawing Game

Pictionary-style game with WebSockets, Canvas drawing, and scalable room management

Tech Stack & Tools

Language	TypeScript (full-stack)
Real-time	Socket.io (WebSockets with fallback to polling)
Backend	Node.js + Express.js
Frontend	React.js + HTML Canvas API
Database	Redis (game state, rooms, scores) + PostgreSQL (user accounts)
Auth	JWT + session cookies
Scaling (optional)	Socket.io Redis Adapter (horizontal scaling across multiple servers)
Deployment	Render.com or Railway (supports WebSocket connections)

Key Algorithms & Concepts

- WebSocket Protocol: Full-duplex TCP connection; Socket.io abstracts rooms, namespaces, events
- Event-Driven Architecture: All game events (draw, guess, timer, nextRound) emit/on via Socket.io
- Canvas Stroke Serialization: Encode drawing as array of {x, y, color, size, type} point events
- Delta Compression: Only broadcast draw deltas (new points), not full canvas state
- Room Management: UUID-based room IDs, host controls (kick, start game, word selection)
- Timer Synchronization: Server-authoritative timer; clients display local countdown synced to server
- Fuzzy Matching (optional): Levenshtein distance to accept near-correct guesses
- Flood Fill Algorithm: Bucket fill tool using BFS on canvas pixel data

Build Steps

60. Define Socket.io event protocol: DRAW, GUESS, JOIN_ROOM, START_GAME, ROUND_OVER, SCORE_UPDATE
61. Build game server: room creation, player join/leave, host assignment, word bank
62. Implement the drawing canvas: mouse/touch events → emit DRAW event → broadcast to room
63. Build canvas replay: receive DRAW events → renderStroke() on all clients' canvases
64. Implement guess checking: GUESS event → server validates → broadcast CORRECT or wrong
65. Add timer: server-side countdown, emit TICK every second, ROUND_OVER when reaches 0
66. Add scoring: +100 for first correct guess, decreasing score, drawer gets points per guesser
67. Build lobby UI: room code sharing, player list with ready status, settings panel

68. Add word variants and hint system: reveal one letter every 20 seconds
69. Add drawing tools: pencil, eraser, flood fill, color picker, undo (stroke history)

Learning Resources

Socket.io Docs	https://socket.io/docs/v4
HTML Canvas Tutorial	https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial
Socket.io Redis Adapter	https://socket.io/docs/v4/redis-adapter
Building Multiplayer Games (free blog)	https://www.digitalocean.com/community/tutorials/node-socket-io
Levenshtein Distance JS	https://github.com/gustavo-rodgers/levenshtein-distance
Flood Fill Algorithm	https://en.wikipedia.org/wiki/Flood_fill
Redis for Game State	https://redis.io/docs/data-types/hashes
Render.com WebSocket Support	https://render.com/docs/web-services

Project 8: Banana Trading Company — Complex Trading Game

Real-time trading simulation with user cards, market fluctuations, and anti-exploit logic

Tech Stack & Tools

Language	TypeScript (full-stack)
Frontend	React.js or Next.js + Canvas or SVG for game UI
Backend	Node.js + Express.js + Socket.io (real-time market)
Database	PostgreSQL (users, cards, portfolios, transactions)
Cache	Redis (real-time price feed, leaderboard)
Auth	NextAuth.js or Passport.js (multiple OAuth providers)
Market Simulation	Custom algorithm (Python or Node.js)
Anti-Exploit	Server-side validation, rate limiting, trade audit log
Leaderboard	Redis ZADD/ZRANGE sorted sets for real-time rankings

Key Algorithms & Concepts

- Market Simulation: Geometric Brownian Motion (GBM) model — same as used in Black-Scholes
- Price Formula: $dS = \mu S \cdot dt + \sigma S \cdot dW$ where μ=drift, σ=volatility, dW=Wiener process (random)
- Supply & Demand Engine: Large buy orders increase price, large sell orders decrease price
- Anti-Exploit: Server re-validates every trade against current server price (not client-submitted price)
- Rate Limiting: Token bucket per user — max N trades per minute to prevent bot abuse
- Card Ownership: NFT-like cards as PostgreSQL rows with owner_id — trades are atomic DB transactions
- Leaderboard: Redis sorted set (portfolio value as score) with O(log n) updates
- Event-Driven Price Spikes: Random news events broadcast via Socket.io causing ±20% price jumps

Build Steps

70. Design database: users, cards (user-generated), portfolios, orders, trade_history, news_events
71. Build market simulation engine: GBM price updates every 5 seconds, broadcast via Socket.io
72. Implement card creation system: users submit art + name + rarity → stored with initial price
73. Build order book: limit orders and market orders; server-side matching engine
74. Add anti-exploit validation: price slippage tolerance, duplicate order detection, rate limiting
75. Build portfolio view: real-time P&L, unrealized gains, holdings list with sparklines
76. Implement leaderboard: Redis ZADD on every portfolio value change, ZREVRANGE for top 10

77. Build in-game newspaper: LLM generates fake financial news headlines based on price changes
78. Add stock ownership rules: max ownership % per card to prevent monopolization
79. Build admin panel: view all trades, ban users, inject market events

Learning Resources

Geometric Brownian Motion (GBM)	https://en.wikipedia.org/wiki/Geometric_Brownian_motion
Building a Matching Engine	https://web.archive.org/web/20160331030151/http://web.archive.org/web/20130306103859/http://www.codeproject.com/Articles/1870/Stock-Exchange-Implementation
Socket.io Rooms & Namespaces	https://socket.io/docs/v4/rooms
Redis Sorted Sets	https://redis.io/docs/data-types/sorted-sets
PostgreSQL ACID Transactions	https://www.postgresql.org/docs/current/tutorial-transactions.html
OpenAI API for News Gen	https://platform.openai.com/docs/guides/text-generation
Rate Limiting with Redis	https://redis.io/glossary/rate-limiting
Recharts for Sparklines	https://recharts.org/en-US

Project 9: LLM Minecraft — Strategy Game with AI Units

Strategy game where each unit is powered by a separate LLM instance with dynamic pipelines

Tech Stack & Tools

Language	Python (LLM backend), TypeScript (frontend)
LLM Framework	LangChain or LangGraph (multi-agent orchestration)
LLM APIs	OpenAI GPT-4o-mini (cost-efficient for many agents), Anthropic Claude, or Ollama (local)
Agent Communication	Custom message bus (Redis pub/sub) or LangGraph state graphs
Backend	FastAPI + Python (game engine + LLM orchestration)
Frontend	React.js + HTML Canvas or Phaser.js (game rendering)
Database	PostgreSQL (game state, agent memory)
Memory	LangChain ConversationBufferMemory or vector store for long-term agent memory
Infra	Docker + docker-compose (run multiple LLM worker containers)
Scaling	Celery (async LLM calls, parallel agent decisions)

Key Algorithms & Concepts

- Multi-Agent Architecture: Each unit has its own LLM context window, system prompt defining its 'personality'
- Dynamic Pipeline Design: Unit type (warrior vs. farmer vs. builder) determines which tools/actions are available
- ReAct Framework: Reason → Act → Observe loop — agent reasons about game state, takes action, observes result
- Hierarchical Agents: Commander agent oversees team, delegates goals to unit agents
- Shared World State: All agents read from same game state (PostgreSQL), write actions atomically
- Token Budget Management: Limit each agent's context to control cost; use summarization for older events
- A* Pathfinding: Units navigate the map grid using A* algorithm
- Fog of War: Each agent only sees tiles within its vision radius — limits LLM context to local info

Build Steps

80. Design the game world: 2D grid, tile types (grass, water, mountain, resource), unit types
81. Build the game loop: every N seconds, collect all unit actions, resolve conflicts, update state
82. Implement A* pathfinding: grid graph, heuristic = Manhattan distance, obstacle avoidance
83. Create the LLM agent wrapper: formats game state as text prompt, parses LLM output as action

84. Define the action schema: MOVE(x,y), ATTACK(unit_id), BUILD(structure), GATHER(resource)
85. Build dynamic pipeline: warrior agents get combat tools, farmers get harvest tools, etc.
86. Set up parallel agent execution: Celery tasks run all agents concurrently, collect actions
87. Add agent memory: short-term (recent actions in context), long-term (summarized history in vector DB)
88. Build minimal frontend: canvas grid rendering, real-time game state updates via WebSocket
89. Add observability: log all LLM prompts/responses for debugging and replay

Learning Resources

LangGraph Docs	https://langchain-ai.github.io/langgraph
LangChain Agents Guide	https://python.langchain.com/docs/modules/agents
ReAct Paper (arxiv)	https://arxiv.org/abs/2210.03629
Multi-Agent Systems Overview	https://lilianweng.github.io/posts/2023-06-23-agent
A* Pathfinding Explained	https://www.redblobgames.com/pathfinding/a-star/introduction.html
Phaser.js Getting Started	https://phaser.io/tutorials/getting-started-phaser3
Ollama (local LLMs)	https://ollama.com
OpenAI Pricing Calculator	https://platform.openai.com/docs/models

Project 10: AI Workflow Automation Tool

Low/no-code platform for building generative AI pipelines (like Dify.ai or Zapier + LLMs)

Tech Stack & Tools

Language	TypeScript (frontend + backend)
Frontend	React.js + React Flow (node-based pipeline editor)
Backend	Node.js + Express.js or Fastify
Database	PostgreSQL (workflows, templates, runs)
Job Queue	BullMQ + Redis (async pipeline execution)
LLM APIs	OpenAI, Anthropic, Google Gemini (user-selectable)
Third-party Integrations	Slack API, SendGrid, Notion API, Google Sheets API, HTTP Webhook
Pipeline Storage	JSON serialized DAG stored in PostgreSQL
Auth	NextAuth.js + JWT
Deployment as API	Auto-generated REST endpoint per pipeline (API key auth)
Knowledge Base	pgvector for RAG-enabled nodes
Templates	Pre-built pipelines: summarizer, email drafter, data extractor, etc.

Key Algorithms & Concepts

- DAG (Directed Acyclic Graph): Pipeline is a DAG; topological sort determines execution order
- Topological Sort (Kahn's Algorithm): Determine which nodes can run in parallel vs. sequentially
- Node Types: Trigger, LLM, Transform, Condition (if/else), Loop, HTTP Request, Output
- Variable Interpolation: {{previous_node.output}} syntax injects upstream results into prompts
- Parallel Execution: Independent DAG branches execute concurrently via BullMQ parallel workers
- Retry with Exponential Backoff: Failed LLM calls retry with 1s, 2s, 4s, 8s delays
- Streaming Output: SSE streams LLM node output in real-time as pipeline runs
- Rate Limiting: Per-user token budget and per-minute LLM call limits

Build Steps

90. Build the React Flow canvas: draggable nodes, connectable edges, property sidebar
91. Define node schema: type, inputs, outputs, config (model, prompt, temperature, etc.)
92. Serialize pipeline to JSON DAG on save: nodes + edges → adjacency list
93. Build pipeline executor: topological sort → execute each node in order/parallel
94. Implement the LLM node: takes prompt template + upstream vars → calls LLM → returns output
95. Implement the HTTP Request node: configure method/URL/headers/body → call external API

96. Implement the Condition node: evaluates JS expression on upstream data → routes to branch
97. Add knowledge base node: embed query → search pgvector → inject relevant chunks into LLM
98. Build deployment: one-click → generate unique API endpoint + API key → run pipeline via REST
99. Build template gallery: save pipelines as templates; community templates browsable in sidebar

Learning Resources

React Flow Docs	https://reactflow.dev/docs
Dify.ai (open source, study code)	https://github.com/langgenius/dify
LangFlow (open source, study code)	https://github.com/langflow-ai/langflow
BullMQ Docs	https://docs.bullmq.io
DAG Topological Sort	https://en.wikipedia.org/wiki/Topological_sorting
OpenAI API Reference	https://platform.openai.com/docs/api-reference
Anthropic API Reference	https://docs.anthropic.com/en/api
PostgreSQL pgvector	https://github.com/pgvector/pgvector

Project 11: PR Review Tool — AI-Powered GitHub App

A GitHub App that auto-reviews PRs for bugs, security issues, and code quality using multiple LLMs

Tech Stack & Tools

Language	Node.js + TypeScript or Python
GitHub Integration	GitHub Apps (Octokit.js or PyGithub)
Auth	GitHub App JWT + installation access tokens (not OAuth)
LLM APIs	OpenAI GPT-4o, Anthropic Claude (route by task type)
Diff Parsing	GitHub API: GET /repos/{owner}/{repo}/pulls/{pull_number}/files
Background Jobs	BullMQ or Celery (async PR analysis)
Database	PostgreSQL (repos, reviews, config)
Frontend (optional)	Next.js dashboard for PR review history and configuration
Deployment	Railway, Render, or AWS Lambda (must be public URL for GitHub webhooks)
Webhook Security	HMAC-SHA256 signature validation (X-Hub-Signature-256 header)
Code Analysis	AST parsing (acorn for JS, ast module for Python) + LLM review

Key Algorithms & Concepts

- GitHub Webhooks: GitHub sends HTTP POST to your server on PR events (opened, synchronize, reopened)
- HMAC-SHA256 Validation: Verify webhook payload integrity using shared secret
- Unified Diff Format: Parse +/- lines in diff to identify changed code; ignore context lines
- AST Analysis: Parse code into Abstract Syntax Tree to detect patterns (unused vars, deep nesting)
- Multi-LLM Routing: Route security checks to Claude (better at security), style to GPT-4o-mini (cheaper)
- Inline Review Comments: GitHub API POST /pulls/{id}/reviews with COMMENT position on specific diff lines
- Review Batching: Aggregate all comments into one review submission (avoid spam notifications)
- Incremental Analysis: Only re-analyze changed files on PR updates (not entire codebase)
- Cost Control: Filter out generated files, lock files, and binary files before sending to LLM

Build Steps

100. Register a GitHub App: set permissions (pull_requests: write, contents: read), webhook URL

101. Build webhook receiver: validate HMAC signature → parse event → enqueue PR analysis job
102. Fetch PR diff: GET /pulls/{id}/files → parse unified diff → extract changed lines per file
103. Filter noise: skip package-lock.json, .min.js, images, generated code, files > 500 lines
104. Build analysis prompts: per-file prompt with diff context → ask LLM for bugs/security/quality issues
105. Parse LLM output: extract file, line number, severity, comment text as structured JSON
106. Post review: GitHub API reviews endpoint — batch all comments into single review submission
107. Add multi-LLM routing: security issues → Claude, quick style → GPT-4o-mini, complex logic → GPT-4o
108. Build configuration: repo owners can set rules (.prreviewer.yml) — ignore files, set severity thresholds
109. Build dashboard: PR review history, accepted/rejected suggestions, cost per review metrics

Learning Resources

GitHub Apps Documentation	https://docs.github.com/en/apps/creating-github-apps/about-creating-github-apps/about-creating-github-apps
Octokit.js Docs	https://octokit.github.io/rest.js
GitHub Pull Request Review API	https://docs.github.com/en/rest/pulls/reviews
Webhook Verification	https://docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries
Unified Diff Format	https://www.gnu.org/software/diffutils/manual/html_node/Unified-Format.html
AST Explorer (browser tool)	https://astexplorer.net
Anthropic API Security Analysis	https://docs.anthropic.com/en/docs/about-claude/use-case-guides/code-review-and-security
Building a GitHub Bot (tutorial)	https://probot.github.io/docs/hello-world

PART 3: ESSENTIAL LEARNING RESOURCES MASTER LIST

Free and paid resources organized by category

Python & Backend

CS50P (Harvard)	https://cs50.harvard.edu/python — Free, best Python intro course
FastAPI Full Course	https://testdriven.io/blog/fastapi-crud — Free blog series
Real Python	https://realpython.com — Best Python tutorials (some free, some paid)
Python Official Docs	https://docs.python.org/3/tutorial — Free, complete
Fluent Python (book)	O'Reilly — Best intermediate Python book

JavaScript / TypeScript

JavaScript.info	https://javascript.info — Best free JS guide, covers everything
TypeScript Handbook	https://www.typescriptlang.org/docs/handbook/intro.html — Free, official
Node.js Docs	https://nodejs.org/en/docs — Free, official
The Odin Project	https://www.theodinproject.com — Free full-stack curriculum
You Don't Know JS (book)	https://github.com/getify/You-Dont-Know-JS — Free on GitHub

React & Next.js

React.dev (official)	https://react.dev — Free, best React docs with interactive examples
Next.js Learn	https://nextjs.org/learn — Free official Next.js course
Jack Herrington YouTube	https://www.youtube.com/@jherr — Free advanced React/Next.js
Fireship.io	https://fireship.io — Short, high-quality videos (some free, some paid)

Databases

PostgreSQL Tutorial	https://www.postgresqltutorial.com — Free, comprehensive
Prisma Docs	https://www.prisma.io/docs — Free, excellent ORM for TypeScript
Redis University	https://university.redis.com — Free courses, official certification

Use The Index, Luke	https://use-the-index-luke.com — Free book on SQL performance

AI / ML / LLMs

DeepLearning.AI Short Courses	https://learn.deeplearning.ai — Free, Andrew Ng + AI companies
fast.ai	https://www.fast.ai — Free, practical deep learning from top down
Hugging Face Course	https://huggingface.co/learn — Free, transformers and NLP
LangChain Docs	https://python.langchain.com/docs — Free, official
Andrej Karpathy YouTube	https://www.youtube.com/@AndrejKarpathy — Free, best LLM fundamentals
3Blue1Brown Neural Networks	https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi — Free

DevOps & Cloud

AWS Free Tier + Tutorials	https://aws.amazon.com/getting-started — Free
Docker Official Docs	https://docs.docker.com/get-started — Free
Kubernetes.io Tutorial	https://kubernetes.io/docs/tutorials/kubernetes-basics — Free, interactive
Terraform Getting Started	https://developer.hashicorp.com/terraform/tutorials — Free
GitHub Actions Docs	https://docs.github.com/en/actions — Free, official

Computer Vision

OpenCV Python Tutorials	https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html — Free
PyImageSearch	https://pyimagesearch.com — Best CV tutorials (free + paid)
DeepFace GitHub	https://github.com/serengil/deepface — Free, best pre-built face recognition
CS231n (Stanford)	https://cs231n.stanford.edu — Free lecture notes on CNNs

Algorithms & System Design

NeetCode.io	https://neetcode.io — Free DSA practice with video explanations
LeetCode	https://leetcode.com — Essential for coding interviews (free + paid)

System Design Primer	https://github.com/donnemartin/system-design-primer — Free GitHub repo
Designing Data-Intensive Apps (book)	Martin Kleppmann — Best system design book (paid, worth it)
redblobgames.com	https://www.redblobgames.com — Free, best interactive game algorithm tutorials

Project Management & Soft Skills

How to Build a Great README	https://www.makeareadme.com — Free
Conventional Commits	https://www.conventionalcommits.org — Free, standard for commit messages
GitHub Student Pack	https://education.github.com/pack — Free tools for students
roadmap.sh	https://roadmap.sh — Free visual learning roadmaps for every role

Final Advice

Build in public: push every project to GitHub and write a README. Document your decisions. Share your progress on LinkedIn or Twitter — it compounds. The goal is not just to finish these projects but to understand every decision you made while building them. Good luck.

CS Research Fields: A Comprehensive Guide

An in-depth overview of every major research area in Computer Science

Covering: High-Level Introduction • Key Topics • Open Problems • Research Directions • Resources

---

This guide covers 15 major research fields in Computer Science with enough depth to help you identify where your interests lie. Each field is organized into five layers of depth: a high-level introduction, key topics you'll need to know, major open problems that researchers are actively working on, current research directions, and recommended resources to go deeper.

You already have familiarity with ML/AI, Algorithms & Theory, and Systems — so those sections will feel more like a structured review, while others may open entirely new doors for you.

Machine Learning & Artificial Intelligence

ML/AI is the study of algorithms and statistical models that enable computers to perform tasks without explicit instructions, relying instead on patterns and inference. It is arguably the most active research area in CS today.

Key Topics

- Supervised, unsupervised, and reinforcement learning
- Deep learning: CNNs, RNNs, Transformers, Diffusion models
- Generative AI: LLMs, image generation, multimodal models
- Optimization: gradient descent, Adam, second-order methods
- Probabilistic graphical models and Bayesian methods
- Federated learning and privacy-preserving ML
- AI safety, alignment, and interpretability

? Open Problems

- Why do large neural networks generalize so well? (Generalization theory)
- How can we build AI systems that are reliably aligned with human values?
- Achieving sample efficiency: learning from few examples like humans
- Bridging the gap between neural networks and symbolic reasoning
- Causal reasoning and out-of-distribution generalization
- Understanding emergent capabilities in large models

□ Current Research Directions

- Foundation models: pretraining, fine-tuning, RLHF, RLAIF
- Mechanistic interpretability: reverse-engineering neural networks
- AI for science: protein folding, drug discovery, climate modeling
- Efficient ML: model compression, quantization, knowledge distillation
- Robustness and adversarial machine learning

□ Resources to Go Deeper

- Deep Learning (Goodfellow, Bengio, Courville) — free online
- Pattern Recognition and Machine Learning (Bishop)
- Papers: arXiv cs.LG, NeurIPS, ICML, ICLR, CVPR, ACL
- Courses: CS229 (Stanford), fast.ai, d2l.ai

□ Algorithms & Computational Complexity

This is the mathematical backbone of computer science — studying what problems can be solved, how efficiently they can be solved, and the fundamental limits of computation itself. It is deeply theoretical but has wide practical impact.

□ Key Topics

- Algorithm design: divide & conquer, dynamic programming, greedy, randomized
- Data structures: advanced trees, heaps, hash tables, graphs
- Complexity classes: P, NP, PSPACE, co-NP, #P, and the polynomial hierarchy
- NP-completeness and reductions
- Approximation algorithms for NP-hard problems
- Online algorithms and competitive analysis
- Parameterized complexity and fine-grained complexity

? Open Problems

- P vs NP: the biggest open problem in all of mathematics
- Does derandomization work? Can every randomized algorithm be derandomized?
- Improving best-known algorithms for 3-SAT, graph isomorphism, etc.
- The complexity of matrix multiplication (is it O(n^2)?)
- Proving superlinear circuit lower bounds

□ Current Research Directions

- Fine-grained complexity: proving tight hardness of specific problems
- Sublinear algorithms: solving problems faster than reading all input
- Quantum complexity theory: BQP vs PH, quantum advantage
- Algorithms for massive graphs and streaming data
- Algebraic complexity theory

□ Resources to Go Deeper

- Introduction to Algorithms (CLRS) — the bible of algorithms

- Computational Complexity: A Modern Approach (Arora & Barak)
- Algorithm Design (Kleinberg & Tardos)
- Courses: CS161 (Stanford), 6.006/6.046 (MIT OpenCourseWare)

□ Computer Systems & Architecture

Systems research covers the design and implementation of the foundational software and hardware infrastructure that everything else runs on — operating systems, compilers, storage, networking, and hardware architecture.

□ Key Topics

- Operating systems: scheduling, memory management, concurrency, file systems
- Computer architecture: CPU design, cache hierarchies, pipelining, out-of-order execution
- Compilers: parsing, intermediate representations, optimization passes
- Storage systems: databases, file systems, log-structured storage
- Network protocols and distributed systems fundamentals
- Virtualization and containerization

? Open Problems

- How to build systems that scale predictably to millions of users?
- Energy efficiency: computing under tight power constraints (edge/IoT)
- Security from the ground up: trusted hardware and secure enclaves
- Programming models for heterogeneous systems (CPUs + GPUs + FPGAs)
- Persistent memory (NVM) — bridging the storage/memory divide

□ Current Research Directions

- Domain-specific hardware accelerators (TPUs, NPUs, custom silicon)
- Serverless computing and FaaS architectures
- Storage-compute disaggregation in datacenters
- Operating system redesign for modern hardware (e.g., Redleaf, Theseus)
- Formal verification of systems software

□ Resources to Go Deeper

- Computer Systems: A Programmer's Perspective (CS:APP)
- Operating Systems: Three Easy Pieces (OSTEP) — free online
- The Architecture of Open Source Applications
- Conferences: OSDI, SOSP, ISCA, MICRO, PLDI, ASPLOS

□ Distributed Systems & Cloud Computing

Distributed systems research focuses on building reliable, scalable, and consistent systems out of many unreliable components spread across a network. This field powers the internet, cloud services, and modern data infrastructure.

□ Key Topics

- Consistency models: linearizability, serializability, eventual consistency
- Consensus algorithms: Paxos, Raft, Byzantine fault tolerance
- Distributed transactions and database systems
- Replication, sharding, and partitioning strategies
- The CAP theorem and its practical implications
- Failure models: crash failures, Byzantine failures

? Open Problems

- Reconciling strong consistency with low latency at global scale
- Debugging and observability in distributed systems
- Geo-distributed databases with sub-millisecond latency
- Simplifying the programming model for distributed applications
- Resilience to sophisticated adversarial failures

□ Current Research Directions

- Disaggregated datacenters and resource pooling
- Edge computing and fog computing architectures
- Decentralized systems (blockchains, P2P networks)
- Programmable networks (P4, SmartNICs)
- ML systems: parameter servers, pipeline parallelism, collective communication

□ Resources to Go Deeper

- Designing Data-Intensive Applications (Kleppmann) — highly recommended
- Distributed Systems (van Steen & Tanenbaum) — free online
- The Google papers: Bigtable, Spanner, MapReduce, Chubby
- Conferences: OSDI, SOSP, EuroSys, NSDI, VLDB

□ Databases & Data Management

Database research covers the theory and systems for storing, querying, and managing large amounts of structured and unstructured data. It sits at the intersection of theory, systems, and increasingly ML.

□ Key Topics

- Relational model and SQL query optimization
- Transaction processing: ACID, MVCC, locking protocols
- NoSQL systems: key-value, document, column, graph databases
- Query processing: join algorithms, execution engines, vectorization
- Data warehousing and OLAP systems (Snowflake, DuckDB era)

- Stream processing: Apache Kafka, Flink, Spark Streaming

❓ Open Problems

- Learned database systems: using ML to optimize query plans
- Unifying transaction and analytics (HTAP systems)
- Managing data at the speed of modern NVM hardware
- Automatic physical design: index selection, partitioning, tuning
- Privacy-preserving data analytics at scale

🔬 Current Research Directions

- Learned query optimizers and cardinality estimators
- LLMs for database interaction (natural language to SQL)
- Vector databases for embedding search (FAISS, Pinecone era)
- Lakehouses: merging data lakes with warehouse capabilities
- Approximate query processing for interactive analytics

📚 Resources to Go Deeper

- Database System Concepts (Silberschatz, Korth, Sudarshan)
- Architecture of a Database System (Hellerstein et al.) — free
- Readings in Database Systems (Red Book) — free online
- Conferences: VLDB, SIGMOD, ICDE

🌐 Computer Networks

Networking research focuses on the design, analysis, and engineering of systems that interconnect computers — from the physical layer and protocols to Internet architecture, content delivery, and future network paradigms.

🔑 Key Topics

- Network protocols: TCP/IP stack, BGP, DNS, HTTP/2/3, QUIC
- Congestion control algorithms (CUBIC, BBR, QUIC CC)
- Software-defined networking (SDN) and network function virtualization
- Wireless networks: WiFi, 5G, mmWave, satellite (Starlink)
- Content delivery networks and edge networking
- Network security: DDoS, BGP hijacking, TLS

❓ Open Problems

- Internet routing scalability: BGP is decades old and fragile
- Net neutrality and the economic structure of the internet
- Reliable low-latency networking for AR/VR and autonomous vehicles
- Securing the internet routing infrastructure
- Wireless spectrum efficiency and coexistence

🔬 Current Research Directions

- Programmable networks with P4 and SmartNICs
- In-network computing: putting compute into the network fabric
- Low-Earth orbit (LEO) satellite networking
- Network verification and formal methods
- Video streaming optimization and adaptive bitrate algorithms

Resources to Go Deeper

- Computer Networking: A Top-Down Approach (Kurose & Ross)
- TCP/IP Illustrated (Stevens)
- Conferences: SIGCOMM, NSDI, IMC

Computer Security & Cryptography

Security research spans the theory and practice of protecting systems, data, and communications from adversaries. It includes both breaking systems (offense) and building secure ones (defense), from cryptographic foundations to real-world attacks.

Key Topics

- Cryptography: symmetric/asymmetric encryption, hash functions, MACs, digital signatures
- Public key infrastructure (PKI) and TLS
- Zero-knowledge proofs, homomorphic encryption, MPC
- Software security: memory safety, fuzzing, binary analysis, exploitation
- Web security: XSS, SQL injection, CSRF, OAuth
- Network security: intrusion detection, firewalls, VPNs
- Hardware security: side-channel attacks (Spectre, Meltdown), trusted execution

? Open Problems

- Post-quantum cryptography: migrating the internet before quantum computers arrive
- Usable security: making secure systems people actually use correctly
- Automated vulnerability discovery at scale
- Formal verification of cryptographic implementations
- AI-powered cyberattacks: LLMs writing malware and exploits

Current Research Directions

- Privacy-enhancing technologies: differential privacy, federated learning
- AI security: adversarial examples, model extraction, membership inference
- Blockchain security and smart contract verification
- Hardware-assisted security (Intel SGX, ARM TrustZone, RISC-V PMP)
- Secure multi-party computation for practical applications

Resources to Go Deeper

- Introduction to Modern Cryptography (Katz & Lindell)
- The Web Application Hacker's Handbook

- Conferences: IEEE S&P (Oakland), CCS, USENIX Security, Crypto, Eurocrypt
- Courses: 6.875 (MIT), CS255 (Stanford)

λ Programming Languages & Formal Methods

PL research studies how to design languages, type systems, and tools that help programmers write correct and efficient code. Formal methods focuses on mathematically proving properties of software and hardware.

Key Topics

- Type theory: dependent types, linear types, effect systems
- Operational and denotational semantics
- Program analysis: static analysis, abstract interpretation, dataflow
- Formal verification: Hoare logic, separation logic, model checking
- Proof assistants: Coq, Lean, Isabelle/HOL, Agda
- Compiler correctness and certified compilation (CompCert)
- Concurrency theory: process calculi, session types

? Open Problems

- Scaling formal verification to real-world software
- Type systems expressive enough to catch real bugs without being too burdensome
- Automatic program synthesis from specifications or examples
- Verified systems software (OS kernels, compilers, hypervisors)
- Semantics of concurrent and weak-memory programs

Current Research Directions

- LLM-assisted formal verification and proof generation
- Rust-inspired ownership types for safe systems programming
- Gradual typing: adding types to dynamic languages
- WebAssembly: formal semantics and security of a universal bytecode
- Language-integrated query and DSL design

Resources to Go Deeper

- Types and Programming Languages (Pierce) — the TAPL book
- Software Foundations series (Pierce et al.) — free, in Coq
- Practical Foundations of Mathematics (Taylor)
- Conferences: POPL, PLDI, ICFP, CAV, TACAS

Human-Computer Interaction (HCI)

HCI studies how people interact with computers and how to design systems that are useful, usable, and delightful. It blends CS with cognitive science, design, and social science to center the human in system design.

□ Key Topics

- User interface design and evaluation
- Accessibility: designing for diverse abilities and needs
- Input modalities: touch, voice, gesture, gaze, brain-computer interfaces
- Information visualization and visual analytics
- Computer-supported cooperative work (CSCW)
- Mixed reality: AR/VR interaction design

? Open Problems

- Natural, reliable voice and gesture interfaces
- Designing AI systems that are transparent and explainable to users
- Ethical design: avoiding dark patterns and addictive technology
- Inclusive design at global scale across cultures and abilities
- Haptic and multi-sensory interfaces beyond screens

□ Current Research Directions

- AI-augmented interfaces and intelligent UIs
- Conversational agents and LLM-powered assistants
- Brain-computer interfaces (Neuralink, BCIs for accessibility)
- Social computing and crowdsourcing platforms
- Programmable matter and shape-changing interfaces

□ Resources to Go Deeper

- Designing with the Mind in Mind (Johnson)
- The Design of Everyday Things (Norman)
- Conferences: CHI, UIST, CSCW, IUI
- Courses: CS147 (Stanford HCI)

□ Computer Graphics & Vision

Graphics research focuses on synthesizing and rendering visual content computationally, while computer vision focuses on extracting meaning from images and video. They are converging rapidly with the rise of neural rendering.

□ Key Topics

- Rendering: ray tracing, rasterization, global illumination, PBR
- 3D geometry: meshes, point clouds, implicit surfaces (NeRF, SDF)
- Animation: physically-based simulation, character animation, rigging
- Computer vision: object detection, semantic segmentation, depth estimation
- Image/video generation: diffusion models, GANs, video synthesis

- 3D reconstruction: SLAM, structure from motion, MVS

❓ Open Problems

- Real-time photorealistic rendering (ray tracing + AI denoising)
- Robust 3D understanding of scenes from 2D images
- Physics-consistent generative models for video
- Efficient neural scene representations for streaming and editing
- Bridging sim-to-real gap for robotics and autonomous driving

□ Current Research Directions

- Neural radiance fields (NeRF) and Gaussian splatting
- Text-to-3D and image-to-3D generation
- Differentiable rendering for inverse problems
- Foundation models for vision (SAM, DINO, CLIP)
- Embodied AI: vision for robotics and navigation

□ Resources to Go Deeper

- Physically Based Rendering (PBRT) — free online
- Computer Vision: Algorithms and Applications (Szeliski) — free online
- Conferences: SIGGRAPH, CVPR, ICCV, ECCV

∞ Theoretical Computer Science (TCS)

TCS is the most mathematically rigorous branch of CS. It encompasses computability, complexity, information theory, coding theory, combinatorics, and the foundations of computation. If you love math and deep questions, this is your home.

□ Key Topics

- Computability theory: Turing machines, halting problem, undecidability
- Computational complexity (overlaps with algorithms)
- Information theory: entropy, mutual information, channel capacity
- Coding theory: error-correcting codes, list decoding
- Combinatorics and graph theory
- Quantum computing theory: quantum algorithms, complexity, error correction
- Communication complexity and circuit complexity

❓ Open Problems

- P vs NP (the most famous open problem in CS)
- Proving super-polynomial circuit lower bounds (beyond P vs NP)
- The log-rank conjecture in communication complexity
- Unique Games Conjecture and its implications for approximability
- Quantum PCP conjecture
- Matrix multiplication exponent: is omega = 2?

□ Current Research Directions

- Pseudorandomness and derandomization
- Algebraic complexity theory
- Average-case complexity and cryptographic hardness
- Fine-grained complexity (SETH, 3SUM hypothesis)
- Quantum information and quantum complexity theory

□ Resources to Go Deeper

- Introduction to the Theory of Computation (Sipser)
- Computational Complexity: A Modern Approach (Arora & Barak) — free draft
- Elements of Information Theory (Cover & Thomas)
- Conferences: STOC, FOCS, CCC, SODA, TCC

□ Quantum Computing

Quantum computing harnesses quantum mechanical phenomena — superposition, entanglement, and interference — to perform computations that would be impractical or impossible on classical computers. It is both a hardware challenge and a theoretical frontier.

□ Key Topics

- Quantum circuits and gate model computation
- Quantum algorithms: Shor's factoring, Grover's search, HHL
- Quantum error correction: surface codes, stabilizer formalism
- Quantum complexity: BQP, QMA, quantum PCP
- NISQ (Noisy Intermediate-Scale Quantum) devices and variational algorithms
- Quantum simulation of physical systems
- Quantum cryptography: QKD, quantum-resistant cryptography

? Open Problems

- Building fault-tolerant quantum computers with error correction
- Finding new quantum algorithms with exponential classical speedup
- Proving quantum computational advantage for practically useful tasks
- Quantum memory and long coherence times
- The power of quantum ML vs classical ML

□ Current Research Directions

- Near-term quantum advantage on specific problems
- Quantum machine learning (QML)
- Topological qubits (Microsoft's approach)
- Quantum networking and the quantum internet
- Classical simulation of quantum circuits (tensor networks)

□ Resources to Go Deeper

- Quantum Computation and Quantum Information (Nielsen & Chuang) — the standard

- Quantum Computing: An Applied Approach (Hidary)
- Courses: Preskill's lecture notes (Caltech), edX courses
- Conferences: QIP, TQC, workshops at STOC/FOCS

□ Bioinformatics & Computational Biology

This interdisciplinary field applies algorithms, ML, and computational methods to biological problems — from analyzing DNA and protein sequences to modeling entire cells, ecosystems, and drug interactions.

□ Key Topics

- Sequence alignment: dynamic programming, BLAST, multiple alignment
- Genome assembly and annotation
- Protein structure prediction (AlphaFold era)
- Single-cell RNA sequencing analysis
- Phylogenetics and evolutionary modeling
- Drug-target interaction prediction
- Gene regulatory network inference

❓ Open Problems

- Protein function prediction from structure alone
- Understanding the non-coding genome
- Predicting phenotype from genotype (the genotype-phenotype problem)
- Modeling cellular dynamics computationally
- RNA structure prediction and design

□ Current Research Directions

- Foundation models for biology (ESM, AlphaFold3, Evo)
- Spatial transcriptomics and multi-omics integration
- AI-assisted drug design and de novo molecule generation
- Pandemic preparedness: viral evolution modeling
- Brain mapping and connectomics

□ Resources to Go Deeper

- Biological Sequence Analysis (Durbin et al.)
- Bioinformatics Algorithms (Compeau & Pevzner) — free online
- Conferences: RECOMB, ISMB, PSB

□□ Robotics & Autonomous Systems

Robotics research sits at the intersection of mechanical engineering, control theory, computer vision, and AI. It focuses on building systems that sense, reason, and act in the physical world.

□ Key Topics

- Robot kinematics and dynamics
- Motion planning: sampling-based (RRT, PRM), optimization-based
- Simultaneous localization and mapping (SLAM)
- Robot perception: object detection, 6-DoF pose estimation
- Control theory: PID, MPC, LQR
- Human-robot interaction
- Multi-robot coordination

❓ Open Problems

- Robust grasping and manipulation of arbitrary objects
- Learning from demonstration: generalizing from few examples
- Safe autonomy: provably safe behavior in open-world environments
- Long-horizon planning under uncertainty
- Sim-to-real transfer: training in simulation, deploying in the real world

□ Current Research Directions

- Foundation models for robotics (RT-2, OpenVLA, pi0)
- Dexterous manipulation with multi-fingered hands
- Legged locomotion on unstructured terrain
- Autonomous driving: perception, prediction, planning stack
- Soft robotics and bio-inspired design

□ Resources to Go Deeper

- Probabilistic Robotics (Thrun, Burgard, Fox) — free online
- Modern Robotics (Lynch & Park) — free online
- Conferences: ICRA, IROS, CoRL, RSS

□ Natural Language Processing (NLP)

NLP focuses on enabling computers to understand, generate, and reason about human language. It has been transformed by Transformer architectures and large language models, making it one of the hottest research areas today.

□ Key Topics

- Language modeling: n-grams, RNNs, Transformers, LLMs
- Text classification, NER, relation extraction
- Machine translation: seq2seq, attention, multilingual models
- Question answering and reading comprehension
- Dialogue systems and conversational AI
- Sentiment analysis and opinion mining
- Text generation and summarization

❓ Open Problems

- Robust reasoning: LLMs still fail on systematic logical problems
- Grounding language to the physical world
- Low-resource NLP for the thousands of languages without data
- Factuality and hallucination in language models
- Understanding what knowledge LLMs actually encode

□ Current Research Directions

- Multimodal LLMs: language + vision, language + audio
- Efficient inference: speculative decoding, MoE, sparse models
- Retrieval-augmented generation (RAG)
- LLMs as reasoning agents (tool use, planning, multi-step reasoning)
- Evaluation methodology: benchmarks, human evaluation, model-based eval

□ Resources to Go Deeper

- Speech and Language Processing (Jurafsky & Martin) — free online
- Natural Language Processing with Transformers (Tunstall et al.)
- Conferences: ACL, EMNLP, NAACL, COLING

□ How to Choose Your Research Area

Given your interest in ML and fundamental CS, here are some particularly natural paths for you:

- **ML Theory & Foundations:** Combines your ML interest with theoretical CS. Focus on understanding why deep learning works, generalization bounds, and optimization theory. Read papers from Sanjeev Arora, Peter Bartlett, and Sham Kakade. Look at venues like COLT and NeurIPS theory track.
- **Algorithms + ML (Learned Algorithms):** A growing intersection — using ML to design better algorithms, and using algorithmic thinking to understand ML. Involves fine-grained complexity, sublinear algorithms, and data structures.
- **NLP & LLM Research:** If you're drawn to language models and their surprising capabilities, NLP research is extremely active. There is enormous room for work on reasoning, evaluation, and grounding.
- **Systems for ML (MLSys):** Bridges systems research with ML — compilers for neural networks, hardware accelerators, distributed training. Very practical and high-impact. Conferences: MLSys, OSDI.
- **TCS / Quantum Computing:** If you love pure math and deep theoretical questions, TCS and quantum computing offer the most mathematically rich research. P vs NP and quantum complexity are among the deepest open questions in science.

A practical tip: read 2-3 recent papers from each field that interests you most. The writing style, the kinds of problems, and the mathematical tools used will tell you more than any guide can. A field is a good fit when the open problems feel exciting to you, not just intellectually impressive.